CENTRALIZED TRAINING AND DECENTRALIZED EXECUTION OF REINFORCEMENT LEARNING AGENT SYSTEM FOR COOPERATIVE TRAFFIC SIGNAL CONTROL

(TRAFFIXION - SMART TRAFFIC FLOW OPTIMIZATION WITH MULTI AGENT REINFORCEMENT LEARNING)

Group 25-26J-473

Project Proposal Report

Fernandopulle S N – IT22000576

B.Sc. (Hons) in Information Technology Specializing in

Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Declaration

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously publish or written by another person expect where the acknowledgement is made in the text.

ID Number	Student Name	Signature
IT22000576	Fernandopulle S N	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

…….

Signature of the supervisor: Date……...

(Prof Nuwan Kodagoda)

…….

Signature of the Co-supervisor: Date……...

(Ms. Karthiga Rajendran)

Abstract

Urban traffic congestion is a pervasive global challenge, leading to significant economic losses, environmental pollution, and reduced quality of life. Traditional and even modern adaptive traffic signal control systems often operate in isolation, optimizing for local intersection efficiency without regard for the broader network-wide consequences. This siloed approach can lead to the propagation of traffic waves and sub-optimal global performance. To address this systemic issue, this research proposes the design and implementation of a sophisticated Multi-Agent Reinforcement Learning (MARL) system for coordinated, citywide traffic flow optimization.

This component forms the core intelligence of a larger smart traffic management system. It leverages a Centralized Training and Decentralized Execution (CTDE) paradigm, specifically utilizing the Value Decomposition Networks (VDN) architecture. In this framework, each traffic intersection is governed by an autonomous agent. During a centralized training phase, these agents learn collaborative strategies by maximizing a shared, global reward signal that reflects citywide traffic efficiency metrics, such as average waiting time, total travel time, and network throughput. The VDN framework mathematically decomposes this global value function into individual agent contributions, enabling each agent to learn the impact of its actions on the collective goal.

Post-training, agents execute decisions in a decentralized manner, relying solely on their local observations (e.g., current phase, queue lengths, waiting times) and lightweight, structured communication from immediate neighboring agents. This communication, which may include predicted outflows or intended phase durations, allows for proactive coordination without the need for a central, real-time controller, ensuring scalability and robustness. The proposed MARL system will be fully integrated with the Simulation of Urban Mobility (SUMO) for rigorous testing and validation. It will interface with other system components, such as emergency vehicle detection, to receive high-priority alerts and integrate them into its optimization calculus.

The expected outcome is a robust, scalable, and intelligent control system that demonstrably reduces citywide congestion, minimizes average journey times, and enhances the responsiveness of the traffic network to dynamic perturbations, including emergency scenarios. This research contributes a practical implementation of advanced MARL theory to the domain of intelligent transportation systems, paving the way for more adaptive and resilient urban mobility solutions.

Keywords: Multi-Agent Reinforcement Learning (MARL), Value Decomposition Networks (VDN), Centralized Training Decentralized Execution (CTDE), Traffic Flow Optimization, SUMO (Simulation of Urban Mobility), Smart Cities, Intelligent Transportation Systems (ITS), Multi-Agent Systems.

List Of Abbreviations

MARL	Multi-Agent Reinforcement Learning
VDN	Value Decomposition Network
CTDE	Centralized Training, Decentralized Execution
RL	Reinforcement Learning
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
DQN	Deep Q-Network
SUMO	Simulation of Urban Mobility
TraCI	Traffic Control Interface
ITS	Intelligent Transportation System
API	Application Programming Interface
CNN	Convolutional Neural Network
ReLU Rectified Linear Unit	Central Processing Unit
Adam	Adaptive Moment Estimation
MSE	Mean Squared Error
JSON	JavaScript Object Notation

Table 1: List Of Abbreviations

Table of Contents

Declaration 2
Abstract 3
List Of Abbreviations 4
Table of Contents 5
List of Figures 7
List of Tables 7
Introduction 8
- Background and Literature 8
- Advantages of the Proposed System 8
- Research Gap 9
- Research Problem 10

Objectives 10
- Main Objective 10
- Specific Objectives 10

Methodology 11
- Requirement Gathering 11
- Development Methodology 12
- Project Management Methodology (Agile with Sprints) 14
- Feasibility Study 14
  - Knowledge on Tools 15

System Analysis 16
- Software Solution Approach 16
- Tools and Technologies 19

Project Requirements 19
- Functional Requirements 19
- Non-Functional Requirements 20

Project Scope 21
- In-Scope 21
- Out-of-Scope 21

Testing 22
1. Unit Testing 22
2. Integration Testing 22
3. Simulation Testing & Performance Benchmarking 22

Expected Outcomes 22

Timeline 23
Communication Management Plan 23
Commercialization 24
Summary 24
References 24

List of Figures

Figure 1: Global reward and next state

Figure 2: High-level architecture of the proposed system

Figure 3: Centralized Training Phase

Figure 4: Decentralized Execution Phase

List of Tables

Table 1: List Of Abbreviations

Table 2: Research gap

Table 3: Estimated budget plan

Table 4: Tools and technologies

Table 5: Timeline

Introduction

Background and Literature

The application of Artificial Intelligence, particularly Reinforcement Learning, to traffic signal control (TSC) has evolved significantly over the past decade. The journey began with single-agent approaches modeling a single intersection as a Markov Decision Process (MDP). Early work utilized table-based methods like Q-learning, but these struggled with the high-dimensional state spaces of real-world traffic. The breakthrough came with the integration of deep neural networks as function approximators, leading to Deep Q-Networks (DQN). DQN and its variants (Double DQN, Dueling DQN) demonstrated the ability to learn complex policies for a single intersection, outperforming fixed-time and adaptive systems like SCATS.

However, the single-agent perspective is myopic. Research quickly identified that multi-intersection coordination is essential for true network-wide optimization. Initial multi-agent approaches employed independent learners, where each intersection runs its own RL algorithm. While simpler to implement, these methods fail to address the non-stationarity of the environment, often resulting in oscillatory and uncoordinated behavior among agents.

This led to the exploration of cooperative MARL frameworks. A significant branch of research involves agents communicating their states or intentions explicitly. While effective, designing stable communication protocols and dealing with the resulting communication overhead remains a challenge. An alternative and highly promising branch is the CTDE paradigm. In CTDE, the centralization during training alleviates the non-stationarity problem by allowing the learning algorithm to reason about the joint action space and global reward. After training, the learned policies are decentralized, requiring no communication during execution, which is ideal for real-time systems.

The Value Decomposition Network (VDN), introduced by [1], is a seminal CTDE algorithm. VDN operates on the principle that the joint action-value function () can be additively decomposed into individual agent value functions (). The learning process involves training a central critic that learns this decomposition, effectively guiding individual agents to learn how their actions contribute to the global objective. This allows for the emergence of complex cooperative behaviors without explicit communication. Following VDN, the QMIX algorithm [2] provided a more general framework by allowing a monotonic mixing of agent values rather than a strict additive decomposition, leading to improved performance on a wider range of tasks.

In the specific domain of traffic control, [3] provided a comprehensive survey of RL-based TSC, highlighting the trend towards MARL. [4] demonstrated an early application of independent Q-learning for multiple intersections. More recently, [5] applied a CTDE method to coordinate traffic lights, showing significant improvements over independent learning and traditional methods. [6] utilized a graph convolutional network to facilitate communication between agents based on the road network topology. This body of literature firmly establishes CTDE, and VDN specifically, as a state-of-the-art approach for networked traffic signal control problems, providing a strong foundational justification for its selection in this component.

Advantages of the Proposed System

The proposed MARL system with VDN offers significant advantages over existing traffic control methods:

1. Global Optimization through Localized Actions: Unlike isolated controllers or independent learners, the VDN framework enables agents to learn policies that are locally executable but globally optimal. The centralized training process ensures that each agent's actions are chosen with an understanding of their impact on the entire network, leading to coordinated green waves and preventing the bottleneck-shifting phenomenon.

2. Scalability and Practicality of Decentralized Execution: A fully centralized controller for a large city's worth of intersections is a single point of failure and requires immense, low-latency communication infrastructure. By employing the CTDE paradigm, our system achieves the benefits of centralized coordination during the learning phase without requiring it during execution. The deployed system is robust, as each intersection operates independently using its pre-trained policy network.

3. Adaptability to Dynamic Conditions: The MARL agents continuously learn from their environment. This means the system can adapt not only to regular daily patterns (morning rush hour, evening traffic) but also to irregular events like accidents, road closures, or special events that disrupt normal traffic flow, something static systems cannot handle.

4. Data-Driven Efficiency: The system optimizes for a user-defined reward function directly correlated with traffic efficiency (e.g., minimizing total wait time). This is a more direct and potentially more effective approach than optimizing for proxy metrics like throughput at a single intersection.

5. Foundation for Integration: This cooperative multi-agent core can serve as a foundation upon which other components of the broader project can be integrated. For instance, information about a detected emergency vehicle (from Component 3) or pedestrian crossings (from Component 4) can be incorporated into the state space of nearby agents, allowing the MARL system to dynamically re-prioritize and adjust its coordination strategy in real-time to handle these events.

Research Gap

The following table outlines the limitations of existing methods and positions the novelty of the proposed VDN-based approach.

Method	Strengths	Research Gaps	References
Single-Agent RL	Effective for optimizing single intersections; well-understood theory.	Fails to account for multi-agent non-stationarity; leads to sub-optimal global performance as agents operate selfishly.	[4]
Independent Q-Learning (IQL)	Simple to implement; fully decentralized.	Struggles with credit assignment and the non-stationary environment, often failing to converge to cooperative policies.	[5]

Centralized RL	Optimal coordination possible with full global observability.	Computationally intractable for large networks; not practical for real-world deployment due to latency and single-point-of-failure.	[6]
Basic CTDE Methods	Addresses non-stationarity during training; enables decentralized execution.	Often uses simple linear value decomposition or requires extensive manual reward shaping to incentivize cooperation.	[7]
Proposed VDN-based MARL	Novelty: Leverages a neural network to learn a non-linear decomposition of the global value function, leading to more sophisticated and emergent cooperative strategies without complex reward engineering. Integrates this with a high-fidelity SUMO simulation for realistic evaluation.		

Table 2: Research gap

Research Problem

Traditional and even modern adaptive traffic signal systems lack the capability for true, large-scale coordination of vehicle traffic management. While they may optimize individual intersections, they fail to generate the emergent behavior necessary for city-wide traffic flow optimization. The core research problem is the inherent non-stationarity and credit assignment challenge in multi-agent environments: how can multiple autonomous traffic signal agents learn cooperative policies that collectively minimize total network-wide delay, when each agent's learning process is constantly disrupted by the changing policies of every other agent? Existing MARL applications in traffic often rely on simplified simulations or struggle with scalability. Therefore, a significant gap exists in the implementation and empirical evaluation of an advanced MARL algorithm like VDN within a realistic, scalable urban traffic simulation (SUMO) to demonstrate tangible improvements in global traffic metrics.

Objectives

Main Objective

To design, implement, and evaluate a Multi-Agent Reinforcement Learning (MARL) system using the Value Decomposition Networks (VDN) framework under the Centralized Training Decentralized Execution (CTDE) paradigm. This system will enable autonomous agents controlling individual traffic intersections to learn cooperative strategies that minimize total network travel time and congestion, and to execute these strategies efficiently using local observations and lightweight communication with neighboring agents within the SUMO simulation environment.

Specific Objectives

1. Architect the Multi-Agent System: Define the multi-agent extension of the single-agent blueprint from Component 1, specifying the structure for inter-agent communication, the global state representation for training, and the shared reward function.

2. Implement the VDN Algorithm: Develop the core VDN neural network architecture in PyTorch/TensorFlow, including the joint Q-network (Q_{tot}) and the individual agent Q-networks (Q_i), and implement the training loop that leverages global information.

3. Design Inter-Agent Communication Protocol: Establish a lightweight and structured communication protocol for trained agents to exchange critical information (e.g., predicted vehicle outflow, intended phase duration) to enable proactive coordination during decentralized execution.

4. Integrate with SUMO via TraCI: Develop a robust interface between the MARL Python environment and the SUMO simulator using the TraCI API to control multiple intersections simultaneously, retrieve global and local states, and send action commands.

5. Train and Validate the MARL System: Conduct extensive training episodes in SUMO under varying traffic scenarios (low, medium, high density) to allow the agents to learn cooperative policies. Validate the learned policies against baselines (e.g., fixed-time, independent DQN agents).

6. Interface with System Components: Develop APIs to receive priority requests from the Emergency Vehicle Detection component (Component 3) and incorporate pedestrian demand metrics from Component 4, shaping the global reward function to respond to these events.

7. Evaluate Performance: Quantify the performance of the trained MARL system using key metrics such as average trip time, average waiting time, total CO2 emissions, and number of vehicles completed, comparing it rigorously against baseline controllers.

8. Analyze Emergent Behavior: Document and analyze the cooperative strategies (e.g., green waves, congestion anticipation) that emerge from the trained system to provide insights into its decision-making process.

Methodology

The proposed methodology focuses on developing a MARL system using the VDN framework, which follows the Centralized Training and Decentralized Execution (CTDE) paradigm to enable coordination between neighboring intersections. The approach is divided into key phases as follows:

Requirement Gathering

This phase involves defining the precise specifications for the MARL system based on the project's overall goals and the interface requirements from other components. Dataset Collection.

- **Functional Requirements :**

  Gather details from other components. From Component 1: the definition of the local state (s_i), action space (a_i), and local reward (r_i) for a single agent. From Component 3: the format and protocol for emergency vehicle priority alerts. From Component 4: the method for receiving pedestrian queue data.

- **Data Requirements:** Identify the simulation data needed from SUMO via TraCI for both the global state (for training) and local states (for execution). This includes vehicle counts per lane, waiting times, vehicle speeds, and signal phase information.
- **Performance Requirements:** Define target metrics for success, e.g., a >20% reduction in average trip time compared to a fixed-time controller under moderate traffic conditions.
- **Interface Requirements:** Specify the software APIs for communication with SUMO (TraCI) and with the other project components (likely via Python function calls or a lightweight messaging protocol like ZeroMQ).

Development Methodology

The development will be modular, focusing on the following core modules:

1. **Agent Communication Protocol Module**:
   - **Description:** This module defines the language of cooperation. It specifies which part of an agent's local state (e.g., its current signal phase, the number of waiting vehicles on each incoming lane) is shared with its immediate neighbors. The protocol will be lightweight to minimize communication overhead and will operate over a defined network topology.
   - Technologies:
     i. Python data classes (e.g., dataclass or Pydantic) will be used to create structured message objects.
     ii. Socket programming or a simpler inter-process communication (IPC) method will be used for message passing between agent processes during simulation.

2. **VDN Algorithm Core Module**:
   - Description: This is the heart of the system. It comprises:
     i. Agent Networks: Individual Deep Q-Networks (DQNs) for each agent. Each DQN takes the agent's augmented state (local state + received neighbor messages) and outputs Q-values for each possible action (e.g., extend phase, switch phase).
     ii. Mixer Network: A central neural network that takes the Q-values from all agents and combines them into a total Q-value estimate. The key constraint of VDN is that this mixer network must output a sum of individual utility functions.

iii. Centralized Training Loop: This loop uses experience replay buffers. A batch of experiences (state, actions, reward, next state) is sampled. The mixer network's prediction of the total Q-value is compared to a target value derived from the global reward, and the error is used to update all agent networks and the mixer network simultaneously.

- Technologies:
  i. PyTorch for building and training all neural networks.
  ii. The torch.nn module will be used for defining network architectures,
  iii. torch.optim module for optimizers like Adam.

3. **SUMO-TraCI Integration Module**:
   - Description: This module acts as the environment. It is responsible for:
     i. State Extraction: Using TraCI commands to poll the SUMO simulation for the current state of each intersection (vehicle counts, waiting times, signal phase).
     ii. Action Execution: Translating the action chosen by an agent (e.g., an integer representing a phase change) into a TraCI command that changes the traffic light state in SUMO.
     iii. Reward Calculation: Calculating the global reward after each action based on the new state of the simulation.
   - Technologies:
     i. The traci Python library provided by SUMO.
     ii. Python scripts will orchestrate the simulation steps and the interaction with the MARL agents.

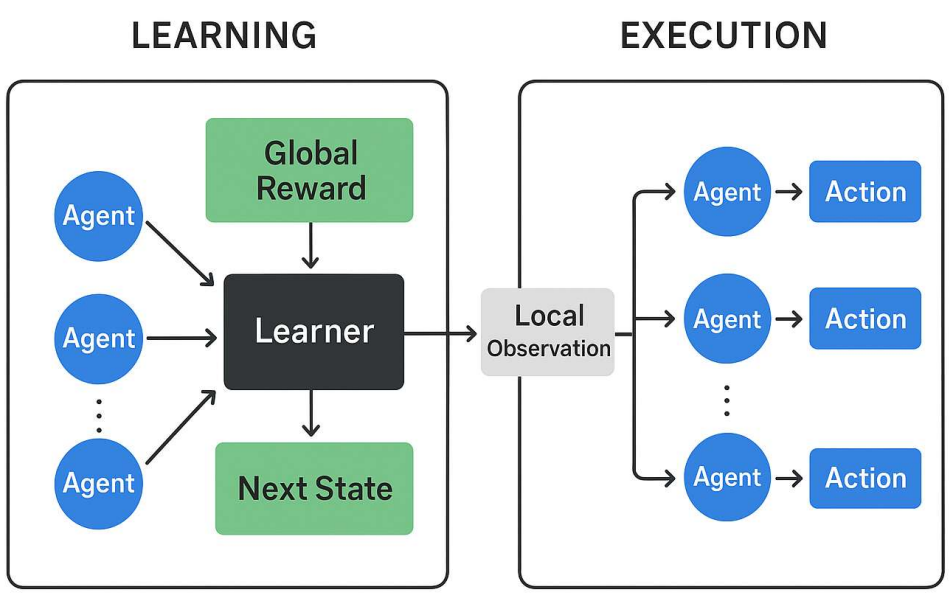

Figure 1: learner aggregates experiences and updates all agent networks using the global reward and next state. During execution (right), each agent uses only its local observation and limited neighbor communication to select an action.

Project Management Methodology (Agile with Sprints)

Research requires effective project management, where scope, time, and cost must be carefully aligned through a structured system or procedure. For the adaptive quiz platform project, a process-oriented methodology will be employed, dividing the project into distinct stages with meticulously planned inputs and outputs. This approach is designed to accommodate changes, identify risks, and manage the project effectively. JIRA will be used to support this methodology, providing a robust platform for tracking progress, managing tasks, and ensuring that project requirements and timelines are met efficiently.

Agile Sprint Structure

1. Sprint 1: Foundation & Protocol Design
    a. Set up the Python & PyTorch development environment.
    b. Finalize the agent state representation and design the inter-agent communication protocol.
2. Sprint 2: Core MARL Infrastructure
    a. Implement the neural network architectures for the DQNs and the VDN mixer.
    b. Build the experience replay buffer and the basic training loop skeleton.
3. Sprint 3: SUMO Integration & Single-Agent Validation
    a. Develop the TraCI interface module for state reading and action execution.
    b. Validate a single agent's learning capability on an isolated intersection.
4. Sprint 4: Multi-Agent Training Loop
    a. Implement the full centralized training loop for multiple agents.
    b. Begin training on a small network (e.g., 2x2 grid) with a simple traffic scenario.
5. Sprint 5: Performance Benchmarking & Analysis
    a. Develop scripts to log key performance metrics (average delay, queue length).
    b. Compare the performance of the MARL system against a fixed-time control baseline.
6. Sprint 6: Advanced Scenario Testing & Optimization
    a. Train and test the system on more complex networks and dynamic traffic scenarios (e.g., peak hour, emergency vehicle priority events triggered by Component 3).
    b. Tune hyperparameters for optimal performance.
7. Sprint 7: Integration & Documentation
    a. Ensure seamless integration with other components (receive input from emergency detection, send data to the dashboard).
    b. Finalize code documentation and prepare the system for demonstration.
8. Sprint 8: Final Evaluation & Report Preparation
    a. Conduct final comprehensive tests and aggregate results.
        i. Document findings, conclusions, and future work for the final project report.

Feasibility Study

Technical Feasibility & Knowledge on Technologies

The project is highly technically feasible. The core technologies are open-source, well-documented, and have a large community support.

- **Python:** The primary language, for which the team has high proficiency.
- **PyTorch:** The chosen deep learning framework is industry-standard and offers excellent flexibility for building custom MARL architectures like VDN.
- **SUMO & TraCI:** While there is a learning curve, SUMO is the de facto standard for academic traffic simulation and provides a powerful and realistic environment. Sufficient time has been allocated to master its API.
- **Reinforcement Learning:** The team has a foundational understanding of RL concepts (MDPs, Q-Learning). The theory behind VDN is well-explained in literature, and its implementation, while non-trivial, is achievable within the project timeline.

Knowledge on Tools

The team is proficient with the essential toolchain:

- **Development:** VS Code, PyCharm, Jupyter Notebook for prototyping.
- **Version Control:** Git and GitHub for collaborative code management.
- **Project Management:** Jira will be used for sprint planning, task assignment, and progress tracking, ensuring adherence to the schedule.
- **Communication:** Microsoft Teams and WhatsApp for daily coordination and problem-solving.

Schedule Feasibility

The 8-sprint plan, spanning approximately 16 weeks, is realistic. The workload is broken down into manageable increments, from foundational setup to complex testing. The Agile approach allows for flexibility to adjust scope based on weekly progress and feedback from supervisors. The dependency on Component 1 is clear: it provides the agent's action and state space definition, which will be finalized early in Sprint 1.

Economic Feasibility

The project requires minimal financial investment, as all software is open-source (Python, PyTorch, SUMO). The primary costs are related to computational resources and documentation:

Estimated Budget:

Type	Cost (LKR)	Remarks
Computational Resources:	10,000	potential cloud GPU credits (Google Colab Pro, AWS EC2) to accelerate the training process for large networks. Training can also

		be performed on a capable local machine.
Miscellaneous	5,000	report printing and documentation.
Total	15,000	-

Table 3: Estimated budget plan

The low-cost structure, combined with the scalability of the system, makes the project economically viable. The solution can later be commercialized or integrated into existing Intelligent Transportation Systems (ITS) for real-world deployment.

System Analysis

Software Solution Approach

1. Project Planning and Architectural Design
    - Requirement Analysis: Detailed functional and non-functional requirements are derived from the core objective. This includes defining the state space (local observations), action space (phase selections), reward structure (local and global components), and the communication protocol between agents.
    - Algorithm Selection: The VDN algorithm is selected for its suitability to the cooperative nature of the problem and its alignment with the Centralized Training and Decentralized Execution (CTDE) paradigm, which is ideal for scalable traffic management.
    - Interface Definition: Clear APIs and data structures are defined for interfacing with Component 1 (the single-agent RL blueprint) to ingest the pre-defined state/action spaces and with the SUMO simulator via TraCI for real-time interaction.

2. Multi-Agent Environment Construction
    - Agent Instantiation: Multiple instances of the RL agent (whose architecture is provided by Component 1) are created, each assigned to control a specific intersection within a SUMO simulation network.
    - Network Graph Definition: The traffic network is abstracted as a graph where nodes represent intersections (agents) and edges represent roads. This graph defines the neighborhood relationships, determining which agents are permitted to communicate with one another.
    - Communication Layer Implementation: A lightweight communication protocol is implemented. This layer allows agents to broadcast a compact signal (e.g., a latent representation of their local traffic state or intended action) to their immediate neighbors within the graph. This signal is the cornerstone of coordination.

3. VDN Framework Development (Centralized Training)
    - Global Reward Reception: During the training phase, a global reward signal is calculated based on the overall performance of the city network. Key metrics

include the average waiting time of all vehicles, the total number of vehicles that have completed their trips, and the overall average speed.

- Value Decomposition: The VDN architecture is implemented. The core of VDN is its neural network that takes the joint action-value function (Q-total) and decomposes it into individual agent value functions (Q-i). The fundamental assumption is that the global Q-function can be additively decomposed into individual contributions: $Q\text{-total}(s, a) \approx \Sigma\ Q\text{-}i(s_i, a_i)$. This decomposition allows each agent to learn how its actions contribute to the global good.
- Centralized Critic Training: A centralized critic network is used during training. This critic has access to the global state and the actions of all agents. It guides the learning process by evaluating the joint actions of all agents against the global reward, providing a gradient to update the parameters of the individual agent networks and the decomposition weights.

4. Policy Learning and Optimization
    - Experience Collection: Agents explore the environment by taking actions (selecting traffic phases) based on their current policy (e.g., ε-greedy). The experiences (state, neighbor signals, action, reward, next state) of all agents are collected in a shared replay buffer.
    - Distributed Learning: Minibatches are sampled from the replay buffer and used to update the parameters of the individual agent networks. The VDN loss function ensures that the sum of the individual Q-values closely approximates the true global Q-value.
    - Policy Refinement: Through millions of simulation steps, the policies of each agent are refined to not only react to their local traffic conditions but also to anticipate and synergize with the actions of their neighboring agents.

5. Decentralized Execution Integration
    - Policy Extraction: Once the training process converges and the policies are sufficiently optimized, the centralized critic is no longer needed. The system transitions to the execution phase.
    - Independent Operation: Each agent now operates entirely independently. It uses its own local observation (e.g., queue lengths, waiting times at its intersection) and the lightweight signals received from its neighbors to compute its action-value function (Q-i).
    - Real-Time Decision Making: The agent selects the action that maximizes its individual Q-value (e.g., using an argmax operation). Because of the way it was trained with VDN, this locally-greedy action is also a optimal action for the global system's benefit.

6. SUMO and TraCI Integration for Simulation
    - Real-Time Control Loop: The trained agents are integrated into the SUMO simulation via the TraCI API. A control loop is established: at every defined time step (e.g., every 5 seconds of simulation time), each agent gathers its local state from SUMO, receives signals from its neighbors, selects an action (traffic light phase), and executes it via TraCI.
    - Scenario Testing: The integrated system is tested under a variety of realistic traffic scenarios, including peak hour congestion, unusual event traffic, and varying traffic flow patterns to evaluate its robustness and performance.

7. Performance Monitoring and Data Logging
    - Metric Collection: A comprehensive logging system is implemented to record key performance indicators (KPIs) at every episode. These include global metrics (total travel time, system-wide average speed) and local metrics (individual intersection queue length, delay).
    - Visualization and Analysis: The logged data is used to generate plots and dashboards (Component 4 may handle this) to visualize the learning progress and the comparative performance against traditional traffic light control methods (e.g., fixed-time, actuated).

The following flowchart illustrates the intricate process of the Centralized Training and Decentralized Execution (CTDE) paradigm using the VDN framework, which forms the core of this component's software approach:

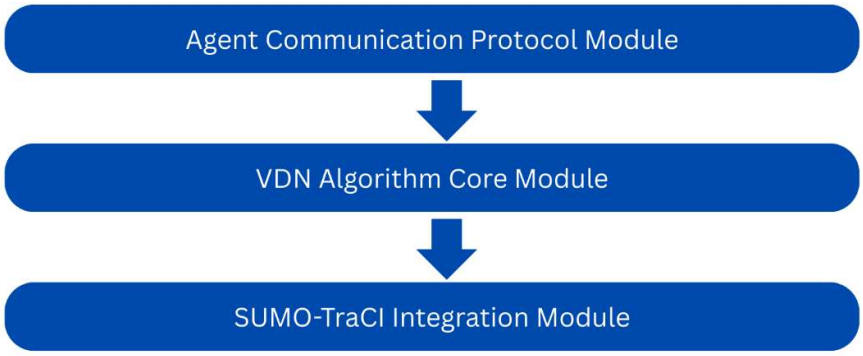

Figure 2: High-level architecture of the proposed system, showing the flow of data between SUMO, the Centralized Trainer, the Individual Agents, and the Communication Module during the different phases.

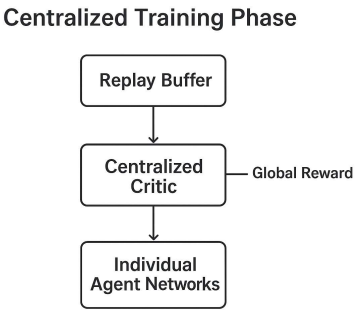

Figure 3: Centralized Training Phase: Experiences from all agents are collected into a shared replay buffer. A centralized critic, with access to the global state and joint actions, computes a global reward and trains the individual agent networks by decomposing the total Q-value using the VDN framework.

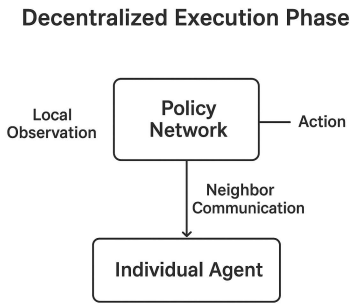

Figure 4: Decentralized Execution Phase: Each trained agent operates independently using only its local observation and lightweight communication from neighbors. It inputs this data into its own policy network to choose the optimal action for its intersection, leading to coordinated, city-wide optimization without a central controller.

Tools and Technologies

Category	Tools & Technologies
Programming Languages	Python 3.x
Frameworks & Libraries	PyTorch, NumPy, Pandas, OpenCV (for potential image-based state), Librosa (N/A for this component), Scikit-learn (for metrics)
Traffic Simulation & Integration	SUMO, TraCI (Python API), libsumo
Data Handling	Custom environment wrapper, JSON for config files
Backend & API	Flask/FastAPI (for potential dashboard connection), ZeroMQ/MPI (for advanced inter-process communication)
Frontend & Visualization	Matplotlib, Seaborn, SUMO-GUI
Development & Project Management	VS Code, Jira, Git/GitHub
Testing Tools	Pytest, SUMO command-line tools for headless testing
Hardware Requirements	GPU (NVIDIA recommended for training), Multi-core CPU, 16GB+ RAM

Table 4: Tools and technologies

Project Requirements

The project requirements define the expected functionality and operational qualities of the *Emergency Vehicle Detection and Triggering Signal in SUMO Simulator* component. These requirements ensure that the system meets both performance goals and real-world applicability.

Functional Requirements

The system shall:

1. **Agent Coordination Mechanism:** Implement a robust communication protocol that allows each RL agent to send and receive compact state or intention signals to and from its predefined neighboring intersection agents.
2. **VDN Algorithm Core:** Implement the Value Decomposition Network architecture, including the neural networks for individual agent Q-functions and the mechanism for decomposing the global Q-value during centralized training.
3. **Centralized Training Loop:** Facilitate a training regime where a centralized critic has access to the global state and joint actions to compute a shared global reward and train the decentralized agent policies.
4. **Decentralized Execution:** Enable each trained agent to operate independently during inference, making phase selection decisions based solely on its local observation and received neighbor signals.
5. **SUMO Integration:** Interface seamlessly with the SUMO simulator via TraCI to:
    - **Read:** Extract the local state for each agent (e.g., queue length per lane, waiting time, number of vehicles).
    - **Write:** Apply the selected action (traffic light phase change) to the corresponding intersection in the simulation.
6. **Performance Logging:** Record comprehensive data from each training episode and evaluation run, including global reward, individual agent rewards, actions taken, communication messages, and standard SUMO output metrics (total travel time, average speed).
7. **Model Persistence:** Provide functionality to save the trained policy network weights of all agents to disk and load them for subsequent evaluation or continued training.

Non-Functional Requirements

The system shall;

1. **Performance:**
    - **Training Efficiency:** The system should be optimized to utilize available GPU resources effectively, aiming to process a high number of simulation steps per hour.
    - **Inference Latency:** During execution, the time between an agent gathering its state and executing an action via TraCI must be minimal (sub-second) to keep pace with the real-time simulation without causing bottlenecks.
2. **Scalability:** The architecture should be designed to function effectively with an increasing number of agents (intersections). The communication protocol should be lightweight to avoid exponential overhead as the network grows.
3. **Robustness:** The system must be able to handle unexpected simulation errors (e.g., TraCI connection drops) gracefully without crashing, logging errors appropriately and attempting to reconnect.

4. **Maintainability:** The codebase shall be modular, well-documented (using docstrings and comments), and adhere to Python style guides (PEP 8). This ensures that the system can be easily understood, debugged, and extended by other developers or researchers in the future.
5. **Reproducibility:** The training process must be deterministic where necessary (e.g., by seeding random number generators) to ensure that experiments can be reproduced exactly, which is a cornerstone of scientific validation.
6. **Usability:** The system should provide clear command-line interfaces or configuration files to easily set parameters such as network topology file paths, training hyperparameters (learning rate, discount factor), and testing scenarios.

Project Scope

The scope of this project defines the boundaries and extent of the *centralized training and decentralized execution of reinforcement learning agent system for cooperative traffic signal controlr* component, outlining the features and functionalities that will be developed (in-scope) and those that will not be included in the current phase (out-of-scope).

In-Scope

The system will include:

- The design and implementation of the multi-agent coordination logic based on the VDN algorithm.
- The development of a lightweight inter-agent communication protocol for exchanging signals.
- Integration with the SUMO simulator for a defined set of urban traffic network topologies (e.g., 4x4 grid).
- Training and evaluation of the MARL system against standard baselines (e.g., fixed-time traffic lights, independent RL agents without coordination).
- Comprehensive analysis of the system's performance impact on city-wide traffic flow metrics.

Out-of-Scope

The system will not include:

- The design of the underlying single-agent RL architecture (this is the responsibility of Component 1).
- The detection and prioritization of emergency vehicles (this is the responsibility of Component 3).
- The modeling and prioritization of pedestrian crossings (this is the responsibility of Component 4).
- Real-world deployment on physical traffic light hardware and infrastructure.
- Integration with Vehicle-to-Everything (V2X) communication systems.

Testing

A rigorous, multi-layered testing strategy will be employed to validate the correctness, performance, and efficacy of the MARL system.

1. **Unit Testing:**
    - **Purpose:** To verify the functionality of individual, isolated components.
    - **Scope:** Tests for the replay buffer (sampling, adding), neural network modules (forward/backward pass), communication protocol (message packing/unpacking), and environment wrapper functions (state extraction from SUMO data).
    - **Tools:** Pytest with mocked objects to simulate dependencies.
2. **Integration Testing:**
    - **Purpose:** To ensure that different modules work together correctly.
    - **Scope:** Testing the full pipeline: SUMO → TraCI → Environment Wrapper → Agent → Action → TraCI → SUMO. Testing the communication between multiple agent instances. Verifying that the centralized training loop correctly updates all networks.
    - **Tools:** Small-scale SUMO networks (e.g., a 2-intersection road) for fast iteration and validation.
3. **Simulation Testing & Performance Benchmarking:**
    - **Purpose:** To evaluate the overall effectiveness and efficiency of the fully integrated system.
    - **Scope:** Running prolonged training sessions on a target network (e.g., 5x5 grid). Evaluating the final trained policies against pre-defined baseline controllers under identical traffic scenarios. Metrics include average travel time, average waiting time, and throughput.
    - **Tools:** Full-scale SUMO simulations, custom data logging scripts, and visualization libraries (Matplotlib) to plot comparative results.

Expected Outcomes

Upon successful completion of this component, the following concrete outcomes are expected:

1. **A Fully Functional MARL Coordination System:** A software library implementing the VDN algorithm for multi-intersection traffic control, complete with training and execution modules.
2. **Quantifiable Performance Improvement:** Empirical results demonstrating a significant reduction in total system-wide travel time (e.g., 15-30%) and average waiting time compared to isolated control methods, proving the value of coordination.
3. **Analysis of Emergent Behavior:** Documentation and analysis of the strategies learned by the agents, such as the creation of "green waves" or coordinated platoon dispersal, providing insights into complex system optimization.

4. **A Reusable Research Framework:** A well-documented codebase that can serve as a foundation for future research, allowing for easy experimentation with different MARL algorithms (e.g., QMIX, MADDPG), communication paradigms, or network topologies.

Timeline

The development will be executed over approximately 16 weeks, structured into Agile sprints. The timeline is designed to prioritize core algorithm development before moving to extensive training and evaluation.

	2025						2026				
Task	July	August	September	October	November	December	January	February	March	April	May
Feasibility study											
Evaluate feasibility and background study											
Requirement gathering											
Task											
Background survey											
Literature review											
Requirement analysis											
Software requirement specification											
Functional, non-functional requirement											
Proposal presentation											
Project proposal report											
PData collection											
ML model creation											
Model training											
Model tunning											
Designing wireframes											
SUMO and Software integration											
Deployment & maintenance											
Progress presentation 1											
Research paper											
Progress presentation 2											
Testing											
Final presentation and viva											

Table 5: Timeline

Communication Management Plan

Regular and structured communication is vital for the success of this complex project. The plan is as follows:

- **Supervisor Meetings:** Weekly sync-ups with the project supervisor to discuss progress, roadblocks, and strategic direction. These will be formal meetings with a prepared agenda and minutes.
- **Inter-Component Meetings:** Bi-weekly meetings with members of Component 1 (Single Agent) and Component 4 (Visualization) to ensure seamless API integration and data flow.
- **Internal Documentation:** Maintenance of a detailed research log/wiki (on GitHub or Confluence) documenting all design decisions, experiment configurations, and results.

- **Progress Reports:** Formal written progress reports submitted at the end of each major sprint, summarizing achievements, metrics, and plans for the next sprint.

Commercialization

The intellectual property generated by this component—a sophisticated algorithm for city-wide traffic coordination—has significant commercial potential. It can be licensed as a software module to:

- **Smart City Solution Providers:** Companies like Siemens, Cisco, or IBM that develop integrated urban management platforms.
- **Traffic Management Software Firms:** Organizations that provide advanced traffic control systems (ATCS) to municipalities.
- **Simulation Software Companies:** Firms like PTV Group (makers of Vissim) could integrate such algorithms into their simulation products for planning and analysis. The value proposition is a proven, AI-driven software solution that reduces congestion and improves urban mobility without the need for expensive physical infrastructure upgrades.

Summary

This proposal outlines the development of a advanced Multi-Agent Reinforcement Learning system to address the critical challenge of citywide traffic congestion. By leveraging the Value Decomposition Network (VDN) framework within a Centralized Training and Decentralized Execution (CTDE) paradigm, this component will enable a network of traffic intersections to collaboratively optimize flow, moving beyond the limitations of isolated control strategies. The project entails the development of a custom MARL algorithm, its integration with the SUMO microscopic traffic simulator, and rigorous testing to validate its performance superiority. The outcome will be a robust, scalable, and intelligent coordination system that forms a core pillar of a modern smart traffic management solution, with strong potential for academic impact and commercial application.

References

[1] Sunehag, P., et al. (2017). "Value-Decomposition Networks For Cooperative Multi-Agent Learning."

[2] Rashid, T., et al. (2018). "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning."

[3] Arel, I., Liu, C., Urbanik, T., & Kohls, A. G. (2010). "Reinforcement learning-based multi-agent system for network traffic signal control."

[4] Khamis, M. A., & Gomaa, W. (2014). "Adaptive multi-objective reinforcement learning for traffic signal control."

[5] Chu, T., Wang, J., Codecà, L., & Li, Z. (2019). "Multi-agent deep reinforcement learning for large-scale traffic signal control."

[6] Cui, Z., Ke, R., & Wang, Y. (2020). "Deep stacked bidirectional LSTM for model-based continuous control in traffic signal coordination."

[7] Bazhenov, V., (2025). “Multi-agent model-based deep reinforcement learning: testing the method in grid-world environments.”